

---

# EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

---

## OPERATION IRON WEB - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

The task and criterion headings in this key are word-for-word identical to `ACT-V-R.md`, so a student can match each criterion one to one.

## Artifact Identity

The instructor-issued artifact hashes are:

```
ACT-V.bin          fab47a6e9f81aaccbc01baffb02ae56d98cc70974e82caa256beb6c52bbb4577
ACT-V.uf2          4eac898720e7fc3663a1b5d2b047318230b98a02f7d7a83a0774cc605dc67278
ACT-V_fixed.bin    795fe72e417cfa28bf61196358eb570c18e7f54f655ca90095b4935b1b362fdc
ACT-V_fixed.uf2    fa0809d6a72561e727790981629eb7028006af03f396b875df9e0e720feb36b1
```

Machine check: `python scripts/verify_ctf.py` returns 10/10 checks passed against the shipped and corrected images. It asserts the four byte pairs, that only those four offsets differ, and the `ACT-V.bin` and `ACT-V_fixed.bin` SHA-256 values. Both `.bin` images are 51,156 bytes and both `.uf2` images are 102,912 bytes.

### The four sabotage sites (summary):

| Defect                 | Function                                                            | File offset         | VA                      | Compromised       | Correct           |
|------------------------|---------------------------------------------------------------------|---------------------|-------------------------|-------------------|-------------------|
| 1 Worm payload         | <code>implant_handle_command</code>                                 | <code>0xA35F</code> | <code>0x1000A35F</code> | <code>0xD1</code> | <code>0xD0</code> |
| 2 Propagation gate     | <code>implant_tick</code> (inlined <code>implant_propagate</code> ) | <code>0xA2F7</code> | <code>0x1000A2F7</code> | <code>0xD1</code> | <code>0xD0</code> |
| 3 Infection marker     | <code>implant_init</code> (inlined <code>implant_infect</code> )    | <code>0xA473</code> | <code>0x1000A473</code> | <code>0xB9</code> | <code>0xB1</code> |
| 4 Tamper authorization | <code>control_handle_frame</code>                                   | <code>0x7569</code> | <code>0x10007569</code> | <code>0xB9</code> | <code>0xB1</code> |

## Task 1: Setup and Initial Analysis (10 points)

### Solution

**Ghidra Setup.** Import `ACT-V.bin` as Raw Binary, language ARM Cortex 32 little endian default, base address `0x10000000`, then run auto-analysis. The Ghidra project name is `IronWeb_Investigation`. Because every defect is a same-size in-place byte patch, the file offset and the VA differ by exactly `0x10000000` ( $VA = offset + 0x10000000$ ).

**Vector Table Decoding.** First 32 bytes of `ACT-V.bin`:

```
00 20 08 20 5D 01 00 10 1B 01 00 10 1D 01 00 10
11 01 00 10 11 01 00 10 11 01 00 10 11 01 00 10
```

| Evidence          | Answer                  |
|-------------------|-------------------------|
| Vector table base | <code>0x10000000</code> |
| Initial SP        | <code>0x20082000</code> |

| Evidence                  | Answer     |
|---------------------------|------------|
| Reset handler (as stored) | 0x1000015D |
| Reset instruction address | 0x1000015C |

The stored reset handler address has bit 0 set, selecting Thumb mode. Clearing bit 0 gives the real entry 0x1000015C.

**Entry and Monitor Loop.** From ACT-V-main-disasm.txt :

```

10000234 <main>:
10000234:    b508          push    {r3, lr}
10000236:    f003 fa93     bl      10003760 <stdio_init_all>
1000023a:    4807          ldr     r0, [pc, #28] @ (10000258 <main+0x24>)
1000023c:    f003 fada     bl      100037f4 <__wrap_puts>
10000240:    f006 f900     bl      10006444 <monitor_init>
10000244:    b110          cbz     r0, 1000024c <main+0x18>
10000246:    f006 f9d7     bl      100065f8 <monitor_step>
1000024a:    e7fc          b.n     10000246 <main+0x12>

```

| Element      | Address    |
|--------------|------------|
| main         | 0x10000234 |
| monitor_init | 0x10006444 |
| monitor_step | 0x100065F8 |

**Module Map.** Anchors for the stripped image:

| Module                         | Anchor function        | Address    |
|--------------------------------|------------------------|------------|
| Entry                          | main                   | 0x10000234 |
| Monitor / tamper state machine | monitor_init           | 0x10006444 |
| Monitor / tamper state machine | monitor_step           | 0x100065F8 |
| Control (sealed tamper path)   | control_handle_frame   | 0x100074FC |
| Tamper authorization           | tamper_auth_apply      | 0x100076AC |
| Latch (actuator)               | latch_apply_command    | 0x100075C0 |
| Latch (actuator)               | latch_fail_safe        | 0x10007634 |
| Implant                        | implant_infected       | 0x1000A2B8 |
| Implant                        | implant_tick           | 0x1000A2CC |
| Implant                        | implant_handle_command | 0x1000A358 |
| Implant                        | implant_init           | 0x1000A444 |
| Crypto                         | envelope_open_hex      | 0x10007950 |
| Crypto                         | crypto_aead_seal       | 0x100077C4 |

| Module | Anchor function       | Address    |
|--------|-----------------------|------------|
| Crypto | crypto_aead_tag_equal | 0x10007778 |
| Radio  | radio_send_frame      | 0x1000A570 |

## Grading Rubric (1-to-1 Mapping)

| Criterion                                                                                             | Points | Full Credit (Answer Key)                                                                                  |
|-------------------------------------------------------------------------------------------------------|--------|-----------------------------------------------------------------------------------------------------------|
| <b>[DOCUMENT]</b> Ghidra project created with the correct name and settings                           | 2      | Project <code>IronWeb_Investigation</code> , raw binary import                                            |
| <b>[DOCUMENT]</b> Processor configured as ARM Cortex 32 little endian default                         | 2      | Screenshot shows the correct processor                                                                    |
| <b>[DOCUMENT]</b> Base address set to 0x10000000                                                      | 2      | Base <code>0x10000000</code>                                                                              |
| <b>[DOCUMENT]</b> Vector table, initial stack pointer, and reset handler identified                   | 2      | Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015D</code> |
| <b>[DOCUMENT]</b> main and the tamper controller state machine (monitor_step) addresses identified    | 1      | <code>main</code> <code>0x10000234</code> , <code>monitor_step</code> <code>0x100065F8</code>             |
| <b>[DOCUMENT]</b> Module map identifies the latch, control, tamper_auth, implant, and monitor anchors | 1      | At least one correct anchor per module                                                                    |

## Instructor Notes & Assembly

- Confirm the Ghidra import used `Raw Binary`, `ARM Cortex 32 little endian default`, base `0x10000000`, and that auto-analysis completed before any address was read. In the language dialog the student must search `Cortex` and pick the ARM Cortex 32 little endian default entry.
- Accept either the Import Results Summary or the Program Information window as proof of the name, language, and base address.
- The stored reset handler `0x1000015D` is odd because bit 0 selects Thumb; clearing it gives `0x1000015C`.
- Always say `reset handler`, never `reset pointer`.
- The vector table is identical in the compromised and corrected images because no defect touches it.
- The module map is graded on coverage, not on exhaustive function recovery: one correctly named anchor per module is sufficient. `implant_infect` and `implant_propagate` are inlined and have no standalone symbol.

## Task 2: Bug #1 The Worm Payload (20 points)

### Solution

**Locate the branch.** In `implant_handle_command` (starts at `0x1000A358`) the payload gate is at file offset `0xA35F` (VA `0x1000A35F`). The corrected image is:

```

1000a358 <implant_handle_command>:
1000a358: 4b30      ldr     r3, [pc, #192] @ (1000a41c <implant_handle_command+0xc4>)
1000a35a: 781a      ldrb    r2, [r3, #0]
1000a35c: 2a00      cmp     r2, #0
1000a35e: d05b      beq.n   1000a418 <implant_handle_command+0xc0>
1000a360: 2800      cmp     r0, #0
1000a362: d059      beq.n   1000a418 <implant_handle_command+0xc0>
1000a364: 2906      cmp     r1, #6
1000a366: d957      bls.n   1000a418 <implant_handle_command+0xc0>
1000a368: b510      push    {r4, lr}
1000a36a: 2207      movs    r2, #7
1000a36c: b0c0      sub     sp, #256 @ 0x100
1000a36e: 492c      ldr     r1, [pc, #176] @ (1000a420 <implant_handle_command+0xc8>)
1000a370: f000 faa0    bl      1000a8b4 <memcmp>
1000a374: 2800      cmp     r0, #0
1000a376: d14d      bne.n   1000a414 <implant_handle_command+0xbc>

```

**Instruction decode.** `ldr r3, [pc, #192]` loads the payload gate at `0x20013CEC`, and `ldrb r2, [r3, #0]` reads it. The branch at `0x1000A35E` decides whether the handler may run. The correct code ignores the frame when the payload gate is clear, so the branch at `0x1000A35E` must be `beq (0xD0)` to the `0x1000A418` return. When the gate is set, the handler checks the frame pointer, requires at least 7 bytes, loads the `IRONWEB` literal at `0x1000AFD4`, and calls `memcmp` against the 7-byte magic. The condition byte is the high byte at `0x1000A35F`.

| Address    | File offset | Compromised byte | Compromised instruction | Correct byte | Correct instruction |
|------------|-------------|------------------|-------------------------|--------------|---------------------|
| 0x1000A35F | 0xA35F      | 0xD1             | bne.n 0x1000A418        | 0xD0         | beq.n 0x1000A418    |

### Patch.

| File Offset | VA         | Original Bytes | Patched Bytes |
|-------------|------------|----------------|---------------|
| 0xA35F      | 0x1000A35F | 5B D1          | 5B D0         |

**Why the node no longer infects itself.** The worm magic is the 7-byte preamble `IRONWEB`, and one frame is 11 bytes: the magic plus a 4-byte synthetic status body. The handler reads the raw inbound payload before the sealed command path ever sees it, so it never opens an envelope and never needs the cipher. Under the compromised `bne`, the payload gate is inverted: the fall-through infect path is taken when the gate is clear, so simply receiving the magic arms the handler and the node reports itself infected. After the patch, `beq` returns while the gate is clear, so the magic is ignored and the frame cannot arm the node. Because the handler is underneath the protocol, no cryptographic control on the envelope can see or stop the frame; the only fix is the gate itself.

### Grading Rubric (1-to-1 Mapping)

| Criterion                                                                                       | Points | Full Credit (Answer Key)                                                  |
|-------------------------------------------------------------------------------------------------|--------|---------------------------------------------------------------------------|
| <b>[DOCUMENT]</b> Located the worm payload handler branch at 0x1000A35F                         | 5      | Address and function ( <code>implant_handle_command</code> ) identified   |
| <b>[DOCUMENT]</b> Documented the <code>IRONWEB</code> magic frame and the raw payload gate      | 5      | 7-byte <code>IRONWEB</code> magic, 11-byte frame, pre-authentication path |
| <b>[DOCUMENT &amp; PATCH]</b> Patched 0xD1 to 0xD0 so the <code>IRONWEB</code> frame is ignored | 7      | Byte 0xD1 changed to 0xD0                                                 |

| Criterion                                                                                       | Points | Full Credit (Answer Key)                             |
|-------------------------------------------------------------------------------------------------|--------|------------------------------------------------------|
| <b>[DOCUMENT]</b> Explained why the incorrect payload handler turns one frame into an infection | 3      | Receipt arms the handler and the node infects itself |

## Instructor Notes & Assembly

- The condition byte is the high byte at `0xA35F`; the correct halfword is `d05b` for `beq.n` and the compromised halfword is `d15b`, so the on-disk bytes are `5B D0` for the fix and `5B D1` for the compromise.
- `beq` branches when the comparison result is equal (the gate is zero); `bne` branches when it is not equal. The register holds the payload gate, so the semantics are "ignore the frame when the gate is clear".
- The worm magic is `TAMPER_IMPLANT_WORM_MAGIC` (`IRONWEB`), `TAMPER_IMPLANT_WORM_MAGIC_LEN` is `7`, and `TAMPER_IMPLANT_WORM_LEN` is `11`.
- Full credit requires both the byte change and a correct statement of the pre-authentication lesson: the worm is not a cipher break, it is a listener beneath the protocol.

## Task 3: Bug #2 The Propagation Gate (20 points)

### Solution

**Locate the branch.** `implant_tick` starts at `0x1000A2CC` and the propagation gate is at file offset `0xA2F7` (VA `0x1000A2F7`). The corrected image is:

```

1000a2cc <implant_tick>:
1000a2cc:    f04f 2ce0    mov.w    ip, #3758153728 @ 0xe000e000
1000a2d0:    4a1b        ldr      r2, [pc, #108] @ (1000a340 <implant_tick+0x74>)
1000a2d2:    6813        ldr      r3, [r2, #0]
1000a2d4:    3301        adds     r3, #1
1000a2d6:    6013        str      r3, [r2, #0]
1000a2d8:    f8dc 2df0    ldr.w    r2, [ip, #3568] @ 0xdf0
1000a2dc:    0791        lsls     r1, r2, #30
1000a2de:    d104        bne.n    1000a2ea <implant_tick+0x1e>
1000a2e0:    4a18        ldr      r2, [pc, #96] @ (1000a344 <implant_tick+0x78>)
1000a2e2:    7812        ldrb     r2, [r2, #0]
1000a2e4:    2ac7        cmp      r2, #199 @ 0xc7
1000a2e6:    b2d1        uxtb     r1, r2
1000a2e8:    d000        beq.n    1000a2ec <implant_tick+0x20>
1000a2ea:    4770        bx       lr
1000a2ec:    079a        lsls     r2, r3, #30
1000a2ee:    d1fc        bne.n    1000a2ea <implant_tick+0x1e>
1000a2f0:    4a15        ldr      r2, [pc, #84] @ (1000a348 <implant_tick+0x7c>)
1000a2f2:    7812        ldrb     r2, [r2, #0]
1000a2f4:    2a00        cmp      r2, #0
1000a2f6:    d0f8        beq.n    1000a2ea <implant_tick+0x1e>
1000a2f8:    4a14        ldr      r2, [pc, #80] @ (1000a34c <implant_tick+0x80>)
1000a2fa:    7812        ldrb     r2, [r2, #0]
1000a2fc:    2a00        cmp      r2, #0
1000a2fe:    d0f4        beq.n    1000a2ea <implant_tick+0x1e>
1000a300:    b500        push     {lr}

```

**Instruction decode.** The tick counter at `0x200136EC` is advanced, the CoreDebug `DHCSR` at `0xE000EDF0` is read, and a non-zero debug state returns early at `0x1000A2DE`. The reserved-sector marker is tested against `0xC7`; a missing marker returns early at `0x1000A2EA`. The low two bits of the tick counter gate the interval at `0x1000A2EC` (`lsls r2, r3, #30` keeps bits 1 and 0), so the node only considers propagation every 4 ticks. `ldr r2, [pc, #84]`

loads the propagation gate at `0x20013CED`, and the branch at `0x1000A2F6` decides whether the node may re-broadcast. The correct code does nothing when the gate is clear, so the branch at `0x1000A2F6` must be `beq (0xD0)` to the `0x1000A2EA` return. The condition byte is the high byte at `0x1000A2F7`.

| Address                 | File offset         | Compromised byte  | Compromised instruction       | Correct byte      | Correct instruction           |
|-------------------------|---------------------|-------------------|-------------------------------|-------------------|-------------------------------|
| <code>0x1000A2F7</code> | <code>0xA2F7</code> | <code>0xD1</code> | <code>bne.n 0x1000A2EA</code> | <code>0xD0</code> | <code>beq.n 0x1000A2EA</code> |

### Patch.

| File Offset         | VA                      | Original Bytes     | Patched Bytes      |
|---------------------|-------------------------|--------------------|--------------------|
| <code>0xA2F7</code> | <code>0x1000A2F7</code> | <code>F8 D1</code> | <code>F8 D0</code> |

**Why the node no longer propagates.** Under the compromised `bne`, the gate is inverted: the fall-through path is taken when the gate is clear, so on every fourth tick an infected node builds the 11-byte `IRONWEB` frame and emits it over the LoRa mesh link. A peer that receives it runs the same handler, infects itself, and emits the frame again, so one accepted frame walks the ring hop by hop. After the patch, `beq` returns while the gate is clear, so an infected node never re-broadcasts. Propagation is a separate control from the marker: clearing the marker removes the local state, and closing the gate removes the transport that would recreate it on every peer.

## Grading Rubric (1-to-1 Mapping)

| Criterion                                                                                                      | Points | Full Credit (Answer Key)                                                                               |
|----------------------------------------------------------------------------------------------------------------|--------|--------------------------------------------------------------------------------------------------------|
| <b>[DOCUMENT]</b> Located the propagation gate branch at <code>0x1000A2F7</code>                               | 5      | Address and function ( <code>implant_tick</code> , inlined <code>implant_propagate</code> ) identified |
| <b>[DOCUMENT]</b> Documented the mesh re-broadcast every 4 ticks while infected                                | 5      | 4-tick interval, <code>IRONWEB</code> frame to peers                                                   |
| <b>[DOCUMENT &amp; PATCH]</b> Patched <code>0xD1</code> to <code>0xD0</code> so the node does not re-broadcast | 7      | Byte <code>0xD1</code> changed to <code>0xD0</code>                                                    |
| <b>[DOCUMENT]</b> Explained why a worm needs only one accepted frame to seed the mesh                          | 3      | Every peer repeats the frame and becomes a source                                                      |

## Instructor Notes & Assembly

- The condition byte is the high byte at `0xA2F7`; the correct halfword is `d0f8` for `beq.n` and the compromised halfword is `d1f8`, so the on-disk bytes are `F8 D0` for the fix and `F8 D1` for the compromise.
- The propagation interval is `TAMPER_IMPLANT_PROPAGATE_INTERVAL_TICKS ( 4 )`. The low two bits of the tick counter implement the modulo, which is why the test is `lsls r2, r3, #30` rather than a division.
- The gate is at `0x20013CED` ( `g_implant_propagate_gate` ); a second flag at `0x20013CEE` ( `g_implant_propagation` ) is checked immediately after and stays gated in both images.
- The `implant_propagate` path is inlined into `implant_tick`; there is no standalone symbol in the stripped image.
- Full credit requires both the byte change and a correct statement of the contamination lesson: in a mesh every node is a router.

## Task 4: Bug #3 The Infection Marker (20 points)

### Solution

**Locate the branch.** The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A444`). The marker gate is at file offset `0xA473` (VA `0x1000A473`). The corrected image is:

```

1000a444 <implant_init>:
1000a444: 2300      movs    r3, #0
1000a446: 2001      movs    r0, #1
1000a448: b530      push    {r4, r5, lr}
1000a44a: 491b      ldr     r1, [pc, #108] @ (1000a4b8 <implant_init+0x74>)
1000a44c: 4c1b      ldr     r4, [pc, #108] @ (1000a4bc <implant_init+0x78>)
1000a44e: b0c1      sub     sp, #260 @ 0x104
1000a450: 4a1b      ldr     r2, [pc, #108] @ (1000a4c0 <implant_init+0x7c>)
1000a452: 7023      strb    r3, [r4, #0]
1000a454: 4d1b      ldr     r5, [pc, #108] @ (1000a4c4 <implant_init+0x80>)
1000a456: 700b      strb    r3, [r1, #0]
1000a458: 491b      ldr     r1, [pc, #108] @ (1000a4c8 <implant_init+0x84>)
1000a45a: 4c1c      ldr     r4, [pc, #112] @ (1000a4cc <implant_init+0x88>)
1000a45c: 602b      str     r3, [r5, #0]
1000a45e: 7013      strb    r3, [r2, #0]
1000a460: 700b      strb    r3, [r1, #0]
1000a462: 4b1b      ldr     r3, [pc, #108] @ (1000a4d0 <implant_init+0x8c>)
1000a464: 7020      strb    r0, [r4, #0]
1000a466: f893 c000  ldrb.w  ip, [r3]
1000a46a: f1bc 0fc7  cmp.w   ip, #199 @ 0xc7
1000a46e: d01f      beq.n   1000a4b0 <implant_init+0x6c>
1000a470: 7812      ldrb    r2, [r2, #0]
1000a472: b1da      cbz     r2, 1000a4ac <implant_init+0x68>
1000a474: 781b      ldrb    r3, [r3, #0]
1000a476: 2bc7      cmp     r3, #199 @ 0xc7
1000a478: d018      beq.n   1000a4ac <implant_init+0x68>
1000a47a: f3ef 8410  mrs     r4, PRIMASK
1000a47e: b672      cpsid   i
1000a480: 22ff      movs    r2, #255 @ 0xff
1000a482: f10d 0001  add.w   r0, sp, #1
1000a486: 4611      mov     r1, r2
1000a488: f000 fa42  bl      1000a910 <memset>
1000a48c: 23c7      movs    r3, #199 @ 0xc7
1000a48e: f44f 5180  mov.w   r1, #4096 @ 0x1000
1000a492: 4810      ldr     r0, [pc, #64] @ (1000a4d4 <implant_init+0x90>)
1000a494: f88d 3000  strb.w  r3, [sp]
1000a498: f000 fb8a  bl      1000abb0 <__flash_range_erase_veneer>
1000a49c: f44f 7280  mov.w   r2, #256 @ 0x100
1000a4a0: 4669      mov     r1, sp
1000a4a2: 480c      ldr     r0, [pc, #48] @ (1000a4d4 <implant_init+0x90>)
1000a4a4: f000 fb68  bl      1000ab78 <__flash_range_program_veneer>
1000a4a8: f384 8810  msr     PRIMASK, r4
1000a4ac: b041      add     sp, #260 @ 0x104
1000a4ae: bd30      pop     {r4, r5, pc}
1000a4b0: 7008      strb    r0, [r1, #0]
1000a4b2: b041      add     sp, #260 @ 0x104
1000a4b4: bd30      pop     {r4, r5, pc}

```

**Instruction decode.** `ldr r3, [pc, #108]` loads the reserved sector at `0x103FF000` (literal at `0x1000A4D0`), and `cmp.w ip, #199` tests the marker against `0xC7`. If the marker is already present the code branches to `0x1000A4B0` and sets the arming flag at `0x20013CEA`. Otherwise `ldrb r2, [r2, #0]` reads the marker gate at `0x20013CEB`. The correct code writes no marker when the gate is clear, so the branch at `0x1000A472` must be `cbz` (`0xB1`) to the `0x1000A4AC` return. When the gate is set, a second check guards the write, and the Pico SDK flash sequence (`strb.w r3, [sp]` then `flash_range_erase` and `flash_range_program`) writes the marker byte `0xC7` once. The condition byte is the high byte at `0x1000A473`.

| Address    | File offset | Compromised byte | Compromised instruction | Correct byte | Correct instruction |
|------------|-------------|------------------|-------------------------|--------------|---------------------|
| 0x1000A473 | 0xA473      | 0xB9             | cbnz r2, 0x1000A4AC     | 0xB1         | cbz r2, 0x1000A4AC  |

**Patch.**

| File Offset | VA         | Original Bytes | Patched Bytes |
|-------------|------------|----------------|---------------|
| 0xA473      | 0x1000A473 | DA B9          | DA B1         |

**The anti-debug obstacle.** The implant reads CoreDebug DHCSR at 0xE000EDF0 and returns early while a probe is attached, which suppresses both the payload handler and the propagation:

```

1000a2d8:    f8dc 2df0    ldr.w   r2, [ip, #3568] @ 0xdf0
1000a2dc:    0791        lsls    r1, r2, #30
1000a2de:    d104        bne.n   1000a2ea <implant_tick+0x1e>

```

```

1000a378:    f04f 23e0    mov.w   r3, #3758153728 @ 0xe000e000
1000a37c:    f8d3 3df0    ldr.w   r3, [r3, #3568] @ 0xdf0
1000a380:    079b        lsls    r3, r3, #30
1000a382:    d147        bne.n   1000a414 <implant_handle_command+0xbc>

```

The shift keeps bit 1 ( C\_HALT ) and bit 0 ( C\_DEBUGEN ) and discards the rest; a non-zero result means a probe is attached and the path returns early. The same register is read again at 0x1000A31E and 0x1000A3FA to stamp the frame body. The guard is identical in both images, so it is an analysis obstacle, not one of the four graded defects.

**Defeating the anti-debug.** Clear the debug bits in the register as seen by the target, or patch the read in a scratch copy. The register is only a view of debug state, so clearing it makes the attach test see no probe. Show the command sequence, not a fabricated transcript; record what the target actually does:

```

arm-none-eabi-gdb ACT-V.elf
(gdb) target extended-remote /dev/cu.usbmodemXXXX
(gdb) monitor reset halt
(gdb) break implant_init
(gdb) continue
(gdb) set {unsigned int}0xE000EDF0 = 0
(gdb) break *0x1000A4A8
(gdb) continue
(gdb) x/4xb 0x103FF000

```

To observe the boot write on the compromised image, break after the flash program at 0x1000A4A8 ( msr PRIMASK, r4 ) in implant\_init , then read the reserved sector at 0x103FF000 and confirm the first byte is C7 . To observe the payload handler and the propagation, clear the debug bits (or patch the ldr.w at 0x1000A2D8 in a scratch copy to load a zero constant) and let implant\_tick run. The scratch copy is for observation only; the shipped artifact is patched at the defect.

**Why no marker is written.** Under the compromised cbnz , the marker gate is inverted: the write path is taken when the gate is clear, so the first boot writes 0xC7 to 0x103FF000 . After the patch, cbz returns while the gate is clear, so the flash erase/program at 0x1000A498 is never reached and the sector stays blank. The marker is the durable state that reports the node infected and that lets the payload handler act, so this fix and the payload fix in Task 2 close the same loop from both ends. The reserved sector sits outside the program region a firmware reflash writes, which is why the marker survives a reflash and why the gate must be fixed in code, not only erased on the bench.

## Grading Rubric (1-to-1 Mapping)

| Criterion                                                                                      | Points | Full Credit (Answer Key)                                                                            |
|------------------------------------------------------------------------------------------------|--------|-----------------------------------------------------------------------------------------------------|
| <b>[DOCUMENT]</b> Located the infection marker branch at 0x1000A473                            | 5      | Address and inlined <code>implant_init</code> path identified                                       |
| <b>[DOCUMENT]</b> Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB   | 5      | <code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method |
| <b>[DOCUMENT &amp; PATCH]</b> Patched 0xB9 to 0xB1 so no marker is written to 0x103FF000       | 7      | Byte <code>0xB9</code> changed to <code>0xB1</code>                                                 |
| <b>[DOCUMENT]</b> Explained the reserved sector 0x103FF000 and the write-once marker byte 0xC7 | 3      | Marker, reserved sector, write-once first run                                                       |

## Instructor Notes & Assembly

- The infect path is inlined into `implant_init`; there is no standalone `implant_infect` symbol in the stripped image.
- The condition byte is the high byte at `0xA473`; the correct halfword is `b1da` for `cbz` and the compromised halfword is `b9da`, so the on-disk bytes are `DA B1` for the fix and `DA B9` for the compromise.
- The marker byte is `TAMPER_IMPLANT_MARKER_BYTE` (`0xC7`), the reserved sector is `TAMPER_IMPLANT_RESERVE_ADDR` (`0x103FF000`), and the marker gate is at `0x20013CEB`.
- The DHCSR address is `TAMPER_IMPLANT_DHCSR_ADDR` (`0xE000EDF0`); bit 0 is `C_DEBUGEN` and bit 1 is `C_HALT`. The anti-debug is identical in both images, so it is an analysis obstacle, not one of the four graded defects.
- Grade the GDB point on a real command sequence and the correct observed code path, not on a memorized register dump. Accept either clearing the bits with GDB or patching the read in a scratch copy.
- A common failure is patching the shipped artifact at `0xA473` before observing the marker. The order matters: defeat the anti-debug, observe, then patch.

## Task 5: Bug #4 The Tamper Authorization (20 points)

### Solution

**Locate the branch.** In `control_handle_frame` (starts at `0x100074FC`) the authorization branch is at file offset `0x7569` (VA `0x10007569`). The corrected image is:

```

1000755e:    990a        ldr    r1, [sp, #40]    @ 0x28
10007560:    4808        ldr    r0, [pc, #32]    @ (10007584 <control_handle_frame+0x88>)
10007562:    aa06        add    r2, sp, #24
10007564:    f000 f8a2    bl     100076ac <tamper_auth_apply>
10007568:    b128        cbz    r0, 10007576 <control_handle_frame+0x7a>
1000756a:    4a07        ldr    r2, [pc, #28]    @ (10007588 <control_handle_frame+0x8c>)
1000756c:    4b07        ldr    r3, [pc, #28]    @ (1000758c <control_handle_frame+0x90>)
1000756e:    7014        strb   r4, [r2, #0]
10007570:    801d        strh   r5, [r3, #0]
10007572:    b017        add    sp, #92 @ 0x5c
10007574:    bd30        pop    {r4, r5, pc}
10007576:    2000        movs   r0, #0
10007578:    b017        add    sp, #92 @ 0x5c
1000757a:    bd30        pop    {r4, r5, pc}

```

**Instruction decode.** After the sealed frame is opened and the command byte is range-checked, `tamper_auth_apply` verifies the anti-replay sequence window and the authenticated-state tag and returns its authorization verdict in `r0`. `cmp r0, #0` inside `tamper_auth_apply` and the branch at `0x10007568` decide whether the command may reach the applied command and zone. The correct code rejects a failed or replayed authorization, so the branch at `0x10007568` must be `cbz (0xB1)` to the `0x10007576` reject path, which returns zero. Only a true verdict falls through to `strb r4, [r2, #0]` and `strh r5, [r3, #0]`, which write the accepted command at `0x20013CE7` and the zone at `0x20013CDA`. The condition byte is the high byte at `0x10007569`.

| Address    | File offset | Compromised byte | Compromised instruction          | Correct byte | Correct instruction             |
|------------|-------------|------------------|----------------------------------|--------------|---------------------------------|
| 0x10007569 | 0x7569      | 0xB9             | <code>cbnz r0, 0x10007576</code> | 0xB1         | <code>cbz r0, 0x10007576</code> |

### Patch.

| File Offset | VA         | Original Bytes | Patched Bytes |
|-------------|------------|----------------|---------------|
| 0x7569      | 0x10007569 | 28 B9          | 28 B1         |

**Why the command now requires authorization.** Under the compromised `cbnz`, the verdict is inverted: a failed or replayed authorization falls through to the store at `0x1000756E`, while a genuine authorization branches to the reject path and returns zero. After the patch, `cbz` sends a false verdict to the reject path at `0x10007576`, so an unauthenticated command, a forged command, and a replayed captured command all fail before the command byte and zone are applied. A legitimate authorized command still returns true and applies. The rest of the path is correct: the envelope is opened under the field key, the command byte is checked against `TAMPER_COMMAND_ALERT (0x01)`, `TAMPER_COMMAND_ARM (0x02)`, and `TAMPER_COMMAND_SECURE (0x03)`, and the zone is checked against the band `0` to `16`.

## Grading Rubric (1-to-1 Mapping)

| Criterion                                                                                                                          | Points | Full Credit (Answer Key)                                              |
|------------------------------------------------------------------------------------------------------------------------------------|--------|-----------------------------------------------------------------------|
| <b>[DOCUMENT]</b> Located the tamper authorization branch at <code>0x10007569</code>                                               | 5      | Address and function ( <code>control_handle_frame</code> ) identified |
| <b>[DOCUMENT]</b> Documented the authorization verdict inversion and the branch condition                                          | 5      | Reject when the verdict is false                                      |
| <b>[DOCUMENT &amp; PATCH]</b> Patched <code>0xB9</code> to <code>0xB1</code> so unauthenticated and replayed commands are rejected | 7      | Byte <code>0xB9</code> changed to <code>0xB1</code>                   |
| <b>[DOCUMENT]</b> Explained that unauthenticated and replayed tamper commands must be rejected                                     | 3      | The applied command must see only an authorized verdict               |

## Instructor Notes & Assembly

- The condition byte is the high byte at `0x7569`; the correct halfword is `b128` for `cbz` and the compromised halfword is `b928`, so the on-disk bytes are `28 B1` for the fix and `28 B9` for the compromise.
- `tamper_auth_apply` performs the monotonic anti-replay check and the authenticated-state tag, so this branch is the verdict for both freshness and state integrity.
- Full credit requires the inversion explanation: the compromised build accepts a false verdict and rejects a true one.
- Point out that the rest of the tamper command path is correct. Only the verdict seam was broken.

- This is the defect that is a policy defect rather than an implant behavior, and it is the one a defender would fix first in production.

## Task 6: Export and Verify (10 points)

### Solution

**Export.** In Ghidra, File -> Export Program..., choose Binary Format, and save as ACT-V\_fixed.bin. The shipped image is 51,156 bytes.

### Convert.

```
python uf2conv.py ACT-V_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-V_fixed.uf2
```

If uf2conv.py is not in the working directory, use the copy shipped with the project repository. The UF2 for ACT-V is 102,912 bytes.

### Verify.

```
python scripts/verify_ctf.py
```

Expected result:

```
10/10 checks passed
```

**Hardware proof.** Flash ACT-V\_fixed.uf2 in BOOTSEL mode and confirm:

- the reserved sector at 0x103FF000 stays blank after a boot;
- a node that receives the IRONWEB magic no longer infects itself or arms the payload handler;
- an infected node no longer re-broadcasts the worm on the 4-tick interval;
- an unauthenticated command and a replayed captured command are rejected before the command and zone are applied;
- a legitimate authorized command still applies, and the arm remote, the manual arm button, and the fail-safe still behave.

### Summary of all patches.

| # | Bug                      | File Offset | Flash Address | Original Byte | Patched Byte |
|---|--------------------------|-------------|---------------|---------------|--------------|
| 1 | The Worm Payload         | 0xA35F      | 0x1000A35F    | D1            | D0           |
| 2 | The Propagation Gate     | 0xA2F7      | 0x1000A2F7    | D1            | D0           |
| 3 | The Infection Marker     | 0xA473      | 0x1000A473    | B9            | B1           |
| 4 | The Tamper Authorization | 0x7569      | 0x10007569    | B9            | B1           |

**Reflection mapping.** The four defects map to real control-system failures:

| Defect                   | Real-world failure                                                                                                                  |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| The Worm Payload         | A payload listens to the raw link below the authenticated protocol, so no cryptographic control on the envelope can see or stop it. |
| The Propagation Gate     | A compromised controller re-broadcasts the payload to every peer, so one compromised node contaminates the whole network.           |
| The Infection Marker     | A payload writes a durable marker to a reserved sector, so the state that re-arms it survives remediation.                          |
| The Tamper Authorization | An inverted verdict lets an unauthenticated or replayed command change a physical command and zone.                                 |

## Grading Rubric (1-to-1 Mapping)

| Criterion                                                                                         | Points | Full Credit (Answer Key)                           |
|---------------------------------------------------------------------------------------------------|--------|----------------------------------------------------|
| <b>[PATCH]</b> Exported ACT-V_fixed.bin from Ghidra                                               | 2      | Valid patched binary                               |
| <b>[PATCH]</b> Converted to ACT-V_fixed.uf2 with the correct base and family                      | 2      | <code>--base 0x10000000 --family 0xe48bff59</code> |
| <b>[DOCUMENT]</b> scripts/verify_ctf.py passes and hardware proves the correct behavior           | 3      | Verifier passes and the hardware proof is shown    |
| <b>[DOCUMENT]</b> Reflection maps each of the four defects to a real-world control-system failure | 3      | Specific mapping for all four                      |

## Instructor Notes & Assembly

- Confirm the exported image differs from `ACT-V.bin` in exactly the four bytes in the table; `scripts/verify_ctf.py` checks this and the SHA-256 values.
- Confirm the UF2 conversion used base `0x10000000` and family `0xe48bff59`.
- The shipped image is 51,156 bytes; the corrected image must be the same size because every patch is in place.
- Grade the reflection on specificity, not length: each of the four defects should name a concrete control-system consequence.
- Remind students that the anti-debug is not patched out of the shipped artifact; only the four defect bytes change.

## How To Breadboard

| Device                           | Pin on device | Pico 2 GPIO | Notes                                           |
|----------------------------------|---------------|-------------|-------------------------------------------------|
| DHT11 cabinet temperature sensor | DATA          | GP4         | 10 kOhm pull-up to 3.3 V if the module needs it |
| 1602 LCD                         | SDA           | GP2         | I2C1, backpack address <code>0x27</code>        |
| 1602 LCD                         | SCL           | GP3         | I2C1, 100 kHz                                   |

| Device            | Pin on device       | Pico 2 GPIO    | Notes                                                |
|-------------------|---------------------|----------------|------------------------------------------------------|
| 1602 LCD          | VCC / GND           | VBUS 5 V / GND | The backpack needs 5 V, not 3.3 V                    |
| RYLR998           | RX                  | GP8 (Pico TX)  | UART1, 115200, network ID 18                         |
| RYLR998           | TX                  | GP9 (Pico RX)  | UART1                                                |
| IR receiver       | OUT                 | GP5            | VS1838B, internal pull-up enabled                    |
| Servo             | signal              | GP14           | PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND |
| Red LED           | anode               | GP16           | INTRUSION, 220 to 330 ohm to GND                     |
| Yellow LED        | anode               | GP17           | ARMED, 220 to 330 ohm to GND                         |
| Green LED         | anode               | GP18           | SECURE, 220 to 330 ohm to GND                        |
| Manual arm button | leg 1               | GP15           | Internal pull-up; leg 2 to GND, never to 3.3 V       |
| Onboard LED       | built in            | GP25           | Heartbeat                                            |
| Debug Probe       | SWCLK / SWDIO / GND | debug header   | For GDB only                                         |

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

## Complete Grading Summary

| Task         | Title                           | Points     |
|--------------|---------------------------------|------------|
| Task 1       | Setup and Initial Analysis      | 10         |
| Task 2       | Bug #1 The Worm Payload         | 20         |
| Task 3       | Bug #2 The Propagation Gate     | 20         |
| Task 4       | Bug #3 The Infection Marker     | 20         |
| Task 5       | Bug #4 The Tamper Authorization | 20         |
| Task 6       | Export and Verify               | 10         |
| <b>TOTAL</b> |                                 | <b>100</b> |

## Instructor Notes

Safety: Use only the supplied Pico 2, Debug Probe, and firmware. Never connect the exercise to an operational industrial control system, a pharmaceutical network, a building-management system, a public network, a military system, or a third-party device.

## Common Student Mistakes

- Patching the low byte of the branch at `0xA35E`, `0xA2F6`, `0xA472`, or `0x7568` instead of the condition byte at `0xA35F`, `0xA2F7`, `0xA473`, or `0x7569`.
- Reading the payload gate or the propagation gate backwards and believing the corrected build still infects or still re-broadcasts.
- Searching for a standalone `implant_infect` or `implant_propagate` symbol and missing that both are inlined into `implant_init` and `implant_tick`.
- Treating the CoreDebug `DHCSR` anti-debug as a defect and trying to patch it, when it is identical in both images and is an analysis obstacle.
- Patching the shipped artifact before observing the marker write, so the payload is never demonstrated.
- Reversing the authorization explanation: under the compromise the accept path is taken when the verdict is false.
- Confusing `cbz` and `cbnz` on the two clearing gates.
- Forgetting that the fix for the marker is two parts: the patch and the reserved-sector erasure.
- Forgetting the UF2 conversion or using the wrong family flag.
- Fabricating a GDB session instead of showing the command sequence and the real observed code path.

## Partial Credit Guidelines

- Award partial credit for a correct address without the correct byte, or a correct byte without the address.
- Award partial credit for documented before/after bytes without the control-flow explanation, or vice versa.
- Award partial credit for a correct GDB command sequence without a clear statement of the observed code path, or the observation without the commands.
- Award partial credit for a correct anti-debug explanation without a working defeat method, or a working method without the explanation.
- Award partial credit for naming the reserved sector and the marker without the persistence lesson, or the lesson without the addresses.
- Award no credit for patches that alter any byte outside the four documented offsets, and no credit for a fabricated GDB session.

## Appendix: Expected Binary Diff

These offsets are from the compiled image loaded at `0x10000000`.

```
--- ACT-V.bin (compromised)
+++ ACT-V_fixed.bin (corrected)

Offset 0x00007569: B9 -> B1    (cbnz r0, 0x10007576 -> cbz r0, 0x10007576)
Offset 0x0000A2F7: D1 -> D0    (bne.n 0x1000A2EA -> beq.n 0x1000A2EA)
Offset 0x0000A35F: D1 -> D0    (bne.n 0x1000A418 -> beq.n 0x1000A418)
Offset 0x0000A473: B9 -> B1    (cbnz r2, 0x1000A4AC -> cbz r2, 0x1000A4AC)
```

| # | Bug                  | File Offset         | Flash Address           | Original Bytes     | Patched Bytes      |
|---|----------------------|---------------------|-------------------------|--------------------|--------------------|
| 1 | The Worm Payload     | <code>0xA35F</code> | <code>0x1000A35F</code> | <code>5B D1</code> | <code>5B D0</code> |
| 2 | The Propagation Gate | <code>0xA2F7</code> | <code>0x1000A2F7</code> | <code>F8 D1</code> | <code>F8 D0</code> |

| # | Bug                      | File Offset | Flash Address | Original Bytes | Patched Bytes |
|---|--------------------------|-------------|---------------|----------------|---------------|
| 3 | The Infection Marker     | 0xA473      | 0x1000A473    | DA B9          | DA B1         |
| 4 | The Tamper Authorization | 0x7569      | 0x10007569    | 28 B9          | 28 B1         |

Four defects, four changed bytes in four instructions: the worm payload gate, the propagation gate, the infection marker gate, and the authorization verdict. No other byte in either image differs.